Document number	P0302R1
Date	2016-06-23
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Removing Allocator Support in std::function (rev 1)

Abstract

The class template `std::function` has several constructors that take an allocator argument, but the semantics are unclear, and there are technical issues with storing an allocator in a type-erased context and then recovering that allocator later for any allocations needed during copy assignment. Those constructors should be removed.

Changes since P0302R0

- Propose to remove the feature entirely rather than deprecate (and so renamed the paper from *Deprecating Allocator Support in std::function*).
- Note about (lack of) feature test macro.

Discussion

The GCC standard library has never provided those constructors at all. The libc++ implementation declares the constructors but ignores the allocator arguments. The MSVC implementation uses the allocator on construction, but does not reuse the allocator if the target is replaced by assignment, which means allocator propagation is not supported.

There have been a number of issues with allocator support in `std::function`:

- 2385 "`function::assign` allocator argument doesn't make sense" -- The resolution was to remove the ability to specify an allocator when replacing the target. The notes from the discussion in Lenexa indicate support for removing more, as proposed by this paper.
- 2386 "`function::operator=` handles allocators incorrectly" -- Closed as NAD due to implementation concerns.
- 2062 "Effect contradictions w/o no-throw guarantee of `std::function` swaps" -- Still Open, no clear direction.
- 2370 "Operations involving type-erased allocators should not be `noexcept` in `std::function`" -- Currently still Open, related to 2062.
- 2501 "`std::function` requires POCMA/POCCA" -- Still Open.
- 2502 "`std::function` does not use `allocator::construct`" -- Still Open.

The attempts to fix allocator support in `function` using polymorphic memory resources (see `std::experimental::function` in Library Fundamentals TS) are not without their own issues (2527, 2564).

It is clear that allocator support in `std::function` is poorly-specified and the source of implementation divergence. I propose that we remove the relevant constructors.

Since these constructors could not be used portably or reliably, I do not see a need for a feature test macro to indicate their absence.

Technical Specification

Remove the constructors taking allocators from [func.wrap.func].

Edit the class synopsis in [func.wrap.func] to remove constructors taking allocators:

```
// 20.12.12.2.1, construct/copy/destroy:
function() noexcept;
function(nullptr_t) noexcept;
function(const function&);
function(function&&);
template<class F> function(F);

template<class A> function(allocator_arg_t, const A&) noexcept;
template<class A> function(allocator_arg_t, const A&,
nullptr_t) noexcept;
template<class A> function(allocator_arg_t, const A&,
```

```
    const function&);
    template<class A> function(allocator_arg_t, const A&,
    function&&);
    template<class F, class A> function(allocator_arg_t, const A&, F);
```

Edit the class synopsis in [func.wrap.func] to remove the `uses_allocator` partial specialization:

```
  // 20.12.12.2.7, specialized algorithms:
  template <class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&);

  template<class R, class... ArgTypes, class Alloc>
    struct uses_allocator<function<R(ArgTypes...)>, Alloc>
      : true_type { };
}
```

Edit [func.wrap.func.con]:

1 ~~When any `function` constructor that takes a first argument of type `allocator_arg_t` is invoked, the second argument shall have a type that conforms to the requirements for Allocator (Table 17.6.3.5). A copy of the allocator argument is used to allocate memory, if necessary, for the internal data structures of the constructed `function` object.~~

```
function() noexcept;
template <class A> function(allocator_arg_t, const A& a) noexcept;
```

2 *Postconditions:* `!*this`.

```
function(nullptr_t) noexcept;
template <class A> function(allocator_arg_t, const A& a, nullptr_t) noexcept;
```

3 *Postconditions:* `!*this`.

```
function(const function& f);
template <class A> function(allocator_arg_t, const A& a, const function& f);
```

4 *Postconditions:* `!*this` if `!f`; otherwise, `*this` targets a copy of `f.target()`.

5 *Throws:* shall not throw exceptions if `f`'s target is a callable object passed via `reference_wrapper` or a function pointer. Otherwise, may throw `bad_alloc` or any exception thrown by the copy constructor of the stored callable object. *[Note:* Implementations are encouraged to avoid the use of dynamically allocated memory for small callable objects, for example, where `f`'s target is an object holding only a pointer or reference to an object and a member function pointer. *— end note]*

```
function(function&& f);
template <class A> function(allocator_arg_t, const A& a, function&& f);
```

6 *Effects:* If `!f`, `*this` has no target; otherwise, move-constructs the target of `f` into the target of `*this`, leaving `f` in a valid state with an unspecified value.

7 *Throws:* shall not throw exceptions [...]

```
template<class F> function(F f);
template <class F, class A> function(allocator_arg_t, const A& a, F f);
```

8 *Requires:* `F` shall be `CopyConstructible`.

9 *Remarks:* ~~These constructors~~This constructor shall not participate in overload resolution unless `f` is Callable (20.12.12.2) for argument types `ArgTypes...` and return type `R`.

Create a new entry in Annex C with the following content:

C.4.5 Clause 20: General utilities library [diff.cpp14.utilities]

20.13.12
Change: Constructors taking allocators removed.
Rationale: No implementation consensus.
Effect on original feature: Valid C++ 2014 code may fail to compile or may change meaning in this International Standard. Specifically, constructing a `std::function` with an allocator is ill-formed and uses-allocator construction will not pass an allocator to `std::function` constructors in this International Standard.